

Flagship Program: RTL Design & Verification (12 Weeks)

Program Overview

This RTL Design and Verification program is structured with hands-on curriculum to build strong fundamentals in SystemVerilog, synthesizable RTL design, and modern verification—ending with a project: APB-controlled UART (APB-UART) and its verification from linear testbench to UVM.

Prerequisites

- Digital design basics: combinational/sequential logic, FSM concepts.
- Comfortable with basic scripting/CLI usage (recommended).
- Familiarity with Verilog is helpful but not required.

Learning Objectives

By the end of this program, participants will be able to:

- Produce industry-style RTL and verification code in SystemVerilog (readable, reviewable, and maintainable).
- Design synthesizable RTL with clear interfaces, clock/reset strategy, and timing-aware implementation choices.
- Develop verification environments that match common team workflows: layered testbenches, BFM, monitors, scoreboards, and checkers.
- Use assertions and functional coverage to drive verification closure and communicate quality with clear metrics.
- Build a practical UVM environment with reusable components and regression-ready tests.
- Debug like an engineer on a project team: triage failures, isolate root cause, and document fixes.
- Finish with a portfolio-ready project (APB-UART RTL + verification report) that demonstrates end-to-end capability for joining an RTL Design/Verification Team.

Format and Time Commitment

- **Duration:** 12 weeks
- **Time:** 6–8 hours/week
- **Structure:** Short lectures + hands-on exercises + weekly deliverables + project reviews

Tools

- **Simulator:** AMD Xilinx Vivado
- **Editor:** VS Code (recommended)
- **Wave viewer:** integrated simulator GUI or GTKWave

- **Version control:** Git

Assessment and Deliverables

- Weekly check-ins (design/verification tasks)
 - Code reviews focusing on readability, correctness, and reusability
 - Final project: APB-UART RTL + verification report (tests, coverage, known limitations)
-

Course Content (Week-by-Week)

Week 1: SystemVerilog Foundations & Coding Standards

Goal: Learn SV syntax and write maintainable code that scales.

- SystemVerilog language essentials: modules, packages, interfaces (intro)
- Data types: 2-state vs 4-state, packed/unpacked arrays, enums, structs
- Operators, casting, and common pitfalls (X/Z behavior, sizing)
- Control flow: if/else, case, loops
- Procedural blocks: always_comb, always_ff, always_latch, initial
- Coding standards: naming, file structure, reset/clocking rules, avoiding latches

Deliverable: Small SV exercises + a “coding standards checklist” used throughout the program.

Week 2: RTL Design Fundamentals

Goal: Build synthesizable, robust RTL with correct timing intent.

- RTL design principles and synthesizability rules
- Combinational vs sequential logic patterns
- Reset strategies (sync/async), clock enables, safe defaults
- FSM design patterns (Moore/Mealy), state encoding considerations
- Datapath + control separation, parameterization

Deliverable: Implement a small FSM-based RTL block with a clear spec and self-checking sanity tests.

Week 3: Verification Fundamentals (Non-UVM)

Goal: Understand how to prove correctness with structured testbenches.

- Verification strategy: what to test, how to partition scenarios
- Testbench components: driver, monitor, checker/scoreboard (concepts)
- Stimulus generation: directed, randomized, constrained-random (intro)
- Assertions (SVA basics): immediate vs concurrent assertions, common properties

- Coverage fundamentals: code coverage vs functional coverage, coverage-driven closure

Deliverable: A basic self-checking testbench for the Week 2 RTL block (directed + simple random).

Week 4: Protocol Basics — APB and UART

Goal: Understand bus and peripheral behavior well enough to design and verify correctly.

- APB protocol overview (transactions, phases, ready/enable/select semantics)
- APB slave register map basics: readable/writable regs, status, control, interrupts (optional)
- UART overview: framing, baud rate, start/stop bits, parity (optional), oversampling (conceptual)
- UART TX/RX behavior: buffering, busy/ready, error flags (framing/parity)

Deliverable: Written mini-spec for APB-UART: register map, transaction timing, and UART behavior assumptions.

Week 5–6: APB-UART RTL Design (Project Part 1)

Goal: Implement the RTL in a modular, verifiable way.

- Translate spec into block diagram and interfaces
- APB slave: address decode, read/write logic, register behavior
- UART TX path: control, data register/FIFO (simple), shift logic
- UART RX path: sampling strategy and receive status (simplified acceptable)
- Status/interrupt flags (optional, if required by spec)
- Design-for-verification: clean separation of protocol layer vs UART core

Deliverable: Synthesizable APB-UART RTL + basic documentation (register map + interface notes).

Week 7–8: APB-UART Verification (Directed/Layered TB)

Goal: Prove functionality with a high-quality non-UVM environment first.

- Build a reusable APB bus functional model (BFM) or tasks
- Create monitors for APB and UART lines
- Build a scoreboard/reference model for key behaviors (registers, TX/RX data)
- Write directed tests: reset, register read/write, TX happy path, RX happy path
- Add negative/edge tests: invalid addresses, back-to-back transfers, busy conditions

Deliverable: Self-checking test suite with a short verification report (what's tested, what's not, known limitations).

Week 9–12: APB-UART Verification with UVM (Project Part 2)

Goal: Build a scalable UVM environment and drive toward coverage closure.

- UVM architecture: test, env, agent, sequencer, driver, monitor, sequence item
- Build APB agent (active) and UART monitor (passive), or vice-versa depending on scope
- Scoreboard integration and prediction strategies
- Sequences: smoke, regression, randomized stress, corner cases
- Functional coverage model: register accesses, UART frame scenarios, error/edge conditions
- Assertions refinement: protocol checks, illegal state detection, end-to-end properties where useful
- Debug workflows: triage failures, reduce to minimal repro, fix RTL vs fix testbench

Deliverable: UVM-based regression + coverage summary and a final demo of APB-UART passing key tests.

Project Acceptance Criteria (Recommended)

- APB register accesses behave per spec (correct data, side-effects, timing).
- UART TX can transmit bytes correctly for a configured baud.
- UART RX can receive bytes and set status correctly for basic cases.
- Verification includes:
 - A scoreboard/checker (not just waveform inspection)
 - At least one functional coverage model
 - Basic assertions for protocol correctness

Optional Extensions (If Time Allows)

- Parity support (even/odd) and parity error reporting
- Small TX/RX FIFOs
- Interrupt generation (TX empty, RX full, error)
- Lint + CDC-style checks (where applicable)